```
import numpy as np

# Boreal Neuro-Core v2.4 - LUT Generation Script
# Packs Sigmoid(x) and Sigmoid'(x) into a single 32-bit word for FPGA BRAM
# [31:16] = Derivative (Q1.15) | [15:0] = Activation (Q1.15)

def generate_boreal_lut(filename="boreal_lut.mem", entries=1024, x_range=8.0):
    scale = 2**15
    with open(filename, "w") as f:
        f.write("// Boreal Neuro-Core v2.4 ROM Initialization\n")
        f.write(f"// Range: [-{x_range}, {x_range}] mapped to {entries} addresses\n")

        for i in range(entries):
            # Map index to x value
            x = (i / (entries / 2) - 1.0) * x_range

            # Compute Sigmoid and Derivative
            sig = 1.0 / (1.0 + np.exp(-x))
            deriv = sig * (1.0 - sig)

            # Convert to Q1.15 Fixed Point
            sig_fixed = int(np.clip(round(sig * (scale - 1)), 0, scale - 1))
            deriv_fixed = int(np.clip(round(deriv * (scale - 1)), 0, scale - 1))

            # Pack: [Deriv][Sig]
            packed = (deriv_fixed << 16) | sig_fixed
            f.write(f"{packed:08X}\n")

if __name__ == "__main__":
    generate_boreal_lut()
    print("LUT generated: boreal_lut.mem")
```

```
/*
 * Adaptive Phase Tracker for Sleep and Tremor Modes
 * Instead of a fixed delay, it uses zero-crossing detection
 * to estimate frequency (f) and trigger at specific phase offsets.
 */

module boreal_pll_tracker (
    input  wire        clk,          // High speed system clock
    input  wire        rst_n,
    input  wire signed [15:0] signal_in, // Filtered EEG
    input  wire        data_ready,

    output reg         trigger_peak,   // Pulses at phase 0 (Sleep Up-State)
    output reg         trigger_anti    // Pulses at phase 180 (Tremor Cancellation)
);

    reg signed [15:0] prev_sample;
    reg [31:0] period_counter;
    reg [31:0] estimated_period;
    reg [31:0] phase_counter;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            period_counter <= 0;
            estimated_period <= 1000000; // Default fallback
            phase_counter <= 0;
            prev_sample <= 0;
        end else if (data_ready) begin
            period_counter <= period_counter + 1;
            phase_counter <= phase_counter + 1;

            // Zero-Crossing Detection (Positive slope)
            if (prev_sample <= 0 && signal_in > 0) begin
                estimated_period <= period_counter;
                period_counter <= 0;
                phase_counter <= 0; // Reset phase on lock
            end

            prev_sample <= signal_in;

            // Trigger Generation
            // Peak at 0 deg (reset point)
            trigger_peak <= (phase_counter == 0);

            // Anti-phase at 180 deg (half period)
            trigger_anti <= (phase_counter == (estimated_period >> 1));
        end else begin
            trigger_peak <= 0;
            trigger_anti <= 0;
        end
    end

endmodule
```

```
/*
 * Boreal Neuro-Core v2.4 (Corrected Build)
 * Key Features:
 * 1. DC-Blocking High Pass Filter (Pre-Inference)
 * 2. Saturation Arithmetic (Anti-Overflow)
 * 3. Systolic Array Active Inference Engine
 * 4. Pipelined PWM Integration
 */

module boreal_apex_core #(
    parameter ADDR_WIDTH = 10,
    parameter CHANNELS   = 8,
    parameter ALPHA      = 16'h7EB8 // 0.99 for DC Blocking Filter
)(
    input  wire        clk,
    input  wire        rst_n,
    input  wire        emergency_halt_n, // Physical override (Bite-switch)

    // ADS1299 Interface (Simplified)
    input  wire [23:0] raw_adc_in,
    input  wire [2:0]  adc_channel_sel,
    input  wire        adc_data_ready,

    // Motor/Stim Control
    output reg  [7:0]  pwm_out
);

    // --- 1. DC-BLOCKING FILTER & TRUNCATION ---
    // Formula: y[n] = x[n] - x[n-1] + alpha * y[n-1]
    reg  signed [23:0] last_raw_x [0:CHANNELS-1];
    reg  signed [31:0] filter_acc [0:CHANNELS-1];
    wire signed [15:0] filtered_sample; // Truncated to Q1.15

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            for (integer i=0; i<CHANNELS; i=i+1) begin
                last_raw_x[i] <= 24'd0;
                filter_acc[i] <= 32'd0;
            end
        end else if (adc_data_ready) begin
            // IIR High Pass Filter logic
            filter_acc[adc_channel_sel] <= ($signed(raw_adc_in) -
last_raw_x[adc_channel_sel]) +
                                           ((filter_acc[adc_channel_sel] *
$signed(ALPHA)) >>> 15);
            last_raw_x[adc_channel_sel] <= $signed(raw_adc_in);
        end
    end

    // Final output truncated to the 16-bit inference range
    assign filtered_sample = filter_acc[adc_channel_sel][31:16];


    // --- 2. DUAL-PORT BRAM LUT (Sigma and Sigma') ---
    wire [31:0] lut_data;
    reg  [ADDR_WIDTH-1:0] lut_addr;
    // (BRAM Instantiation logic would go here, initialized with boreal_lut.mem)


    // --- 3. ACTIVE INFERENCE ENGINE (MACC with Saturation) ---
    reg signed [15:0] mu [0:CHANNELS-1];     // Internal State
    reg signed [15:0] epsilon [0:CHANNELS-1]; // Prediction Error
    reg signed [15:0] weight [0:CHANNELS-1];  // Weight fetched from BRAM

    localparam signed MAX_VAL = 16'h7FFF;
    localparam signed MIN_VAL = 16'h8000;
```

```
    // Gradient Update: mu = mu + eta*(epsilon * deriv - lambda*mu)
    // eta and lambda are implemented as bit-shifts for latency closure
    wire signed [31:0] grad_product = epsilon[adc_channel_sel] *
$signed(lut_data[31:16]); // eps * sigma'
    wire signed [15:0] decay = mu[adc_channel_sel] >>> 4; // lambda = 1/16

    always @(posedge clk) begin
        if (!emergency_halt_n) begin
            for (integer i=0; i<CHANNELS; i=i+1) mu[i] <= 16'd0;
        end else if (adc_data_ready) begin
            // Prediction Error: y - sigma(W*mu)
            epsilon[adc_channel_sel] <= filtered_sample - $signed(lut_data[15:0]);

            // Saturation Arithmetic on State Update
            if (grad_product[31:16] > MAX_VAL)
                mu[adc_channel_sel] <= MAX_VAL;
            else if (grad_product[31:16] < MIN_VAL)
                mu[adc_channel_sel] <= MIN_VAL;
            else
                mu[adc_channel_sel] <= mu[adc_channel_sel] + grad_product[20:5] - decay;
        end
    end


    // --- 4. INTEGRATED PWM OUTPUT ---
    reg [23:0] pwm_accumulator;
    always @(posedge clk) begin
        if (!emergency_halt_n) begin
            pwm_accumulator <= 24'd0;
            pwm_out <= 8'd0;
        end else begin
            // Integrator: PWM = Integral(mu * Kp)
            pwm_accumulator <= pwm_accumulator + $signed(mu[0]); // Using Channel 0 for
main control
            pwm_out <= pwm_accumulator[23:16];
        end
    end

endmodule
```

```
/*
 * Dual-Port BRAM for Boreal Neuro-Core
 * Port A: Read-only for Weight Streaming & Activation LUT
 * Port B: Write-access for Active Inference Weight Updates (Learning)
 */

module boreal_memory #(
    parameter ADDR_WIDTH = 10,
    parameter DATA_WIDTH = 32
)(
    input  wire                  clk,
    // Port A (Inference Read)
    input  wire [ADDR_WIDTH-1:0] addr_a,
    output reg  [DATA_WIDTH-1:0] dout_a,

    // Port B (Learning Write/Update)
    input  wire                  we_b,
    input  wire [ADDR_WIDTH-1:0] addr_b,
    input  wire [DATA_WIDTH-1:0] din_b,
    output reg  [DATA_WIDTH-1:0] dout_b
);

    // Memory array
    reg [DATA_WIDTH-1:0] ram [0:(2**ADDR_WIDTH)-1];

    // Initialization (Load the .mem file generated by Python)
    initial begin
        $readmemh("boreal_lut.mem", ram);
    end

    // Port A: Synchronous Read
    always @(posedge clk) begin
        dout_a <= ram[addr_a];
    end

    // Port B: Synchronous Read/Write
    always @(posedge clk) begin
        if (we_b)
            ram[addr_b] <= din_b;
        dout_b <= ram[addr_b];
    end

endmodule
```

```
/*
 * Boreal Neuro-Core v2.4 - Top Level Wrapper
 * Integrates ADC Interface, DC-Blockers, Inference Engine, and PLL
 */

module boreal_system_top (
    input  wire clk_100m,
    input  wire rst_n,
    input  wire bite_switch_n, // Hardware Override

    // ADS1299 SPI Physical
    input  wire ads_drdy_n,
    input  wire ads_miso,
    output wire ads_sclk,
    output wire ads_cs_n,

    // Physical Outputs
    output wire pwm_motor_out,
    output wire stim_trigger_out
);

    // Internal Interconnects
    wire [127:0] raw_channels;
    wire         adc_valid;
    wire [7:0]   system_pwm;
    wire         peak_detect;
    wire         anti_detect;

    // 1. SPI Ingestion Logic (From previous turn)
    boreal_ads1299_spi ads_interface (
        .clk_100m(clk_100m),
        .rst_n(rst_n),
        .bite_switch_n(bite_switch_n),
        .drdy_n(ads_drdy_n),
        .miso(ads_miso),
        .sclk(ads_sclk),
        .cs_n(ads_cs_n),
        .eeg_channels_out(raw_channels),
        .data_valid(adc_valid)
    );

    // 2. The Core Active Inference Engine
    // Processes all 8 channels with DC-blocking and Saturation Arithmetic
    boreal_apex_core inference_engine (
        .clk(clk_100m),
        .rst_n(rst_n),
        .emergency_halt_n(bite_switch_n),
        .raw_adc_in(raw_channels[23:0]), // Channel 0 feed
        .adc_channel_sel(3'b000),
        .adc_data_ready(adc_valid),
        .pwm_out(system_pwm)
    );

    // 3. Adaptive Phase Tracking (For Closed-Loop Stimulation)
    boreal_pll_tracker pll (
        .clk(clk_100m),
        .rst_n(rst_n),
        .signal_in(inference_engine.filtered_sample),
        .data_ready(adc_valid),
        .trigger_peak(peak_detect),
        .trigger_anti(anti_detect)
    );

    // 4. Output Mapping
    assign pwm_motor_out = (system_pwm > 128); // Simple 1-bit PWM for demo
    assign stim_trigger_out = peak_detect;      // Fire on Delta Peak (Sleep Mode)
```

```
endmodule
```